

LAPORAN MODUL 1

PRAKTIKUM PERMODELAN STATISTIKA



Disusun oleh :

Nama : IRVAN CAHYA NUGRAHA
NIM : 245410034
Kelas : IF-1

PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024/2025

LAPORAN PRAKTIKUM

MODUL 1

A. PEMBAHASAN LISTING

Praktik

1. Praktik 1

```
[10]
✓ 0 d      print ('Hello selamat belajar pemodelan Statistika')

[1] "Hello selamat belajar pemodelan Statistika"

[11]
✓ 1m      # Menginstal beberapa pustaka
install.packages(c("dplyr", "ggplot2", "caret"))
library(dplyr)
library(ggplot2)
library(caret)

Installing packages into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

[12]
✓ 0 d      # Membuat vektor numerik
nilai <- c(80, 85, 90, 95, 100)
cat("Vektor nilai:", nilai, "\n")

# Membuat vektor karakter
nama <- c("Ali", "Budi", "Cici")
cat("Vektor nama:", nama, "\n")

# Membuat vektor logis
lulus <- c(TRUE, TRUE, FALSE)
cat("Vektor lulus:", lulus, "\n")

# Menggunakan seq() untuk urutan
urutan <- seq(from = 1, to = 10, by = 2)
cat("Vektor urutan:", urutan, "\n")

# Menggunakan rep() untuk pengulangan
ulang <- rep(5, times = 3)
cat("Vektor pengulangan:", ulang, "\n")

Vektor nilai: 80 85 90 95 100
Vektor nama: Ali Budi Cici
Vektor lulus: TRUE TRUE FALSE
Vektor urutan: 1 3 5 7 9
Vektor pengulangan: 5 5 5

[13]
✓ 0 d      # Operasi aritmatika
nilai <- c(80, 85, 90, 95, 100)
nilai_tambah <- nilai + 5 # Menambah 5 ke setiap elemen
cat("Nilai setelah ditambah 5:", nilai_tambah, "\n")

# Operasi antar vektor
bonus <- c(1, 2, 3, 4, 5)
total <- nilai + bonus
cat("Total nilai dengan bonus:", total, "\n")

# Operasi logis
tinggi <- nilai > 85
cat("Nilai > 85:", tinggi, "\n")

Nilai setelah ditambah 5: 85 90 95 100 105
Total nilai dengan bonus: 81 87 93 99 105
Nilai > 85: FALSE FALSE TRUE TRUE TRUE
```

```
# Pengindeksan
nilai <- c(80, 85, 90, 95, 100)
cat("Elemen kedua:", nilai[2], "\n")
cat("Elemen 1 sampai 3:", nilai[1:3], "\n")

# Filtering
nilai_tinggi <- nilai[nilai > 85]
cat("Nilai > 85:", nilai_tinggi, "\n")
```

→ Elemen kedua: 85
Elemen 1 sampai 3: 80 85 90
Nilai > 85: 90 95 100

Pembahasan : A. Membuat dan Memanipulasi Vektor

Praktik 1: Membuat Vektor dengan `c()`, `seq()`, dan `rep()`

Program di atas adalah program R yang membuat vektor dengan fungsi `c()`, `seq()`, dan `rep()`. Pada bagian `nilai <- c(80, 85, 90, 95, 100)` dibuat sebuah vektor numerik bernama `nilai` yang berisi lima angka yaitu 80, 85, 90, 95, dan 100. Ketika dijalankan, fungsi `cat()` akan menampilkan teks *Vektor nilai* diikuti isi dari vektor `nilai`. Selanjutnya dibuat vektor karakter dengan `nama <- c("Ali", "Budi", "Cici")` yang menyimpan tiga data string, kemudian dibuat juga vektor logis dengan `lulus <- c(TRUE, TRUE, FALSE)` yang menyimpan nilai logika `TRUE` dan `FALSE`. Pada baris berikutnya digunakan `seq(from = 1, to = 10, by = 2)` yang menghasilkan urutan angka dari 1 sampai 10 dengan jarak 2 sehingga hasilnya 1, 3, 5, 7, 9. Terakhir digunakan `rep(5, times = 3)` yang menghasilkan pengulangan angka 5 sebanyak 3 kali. Semua hasil tersebut ditampilkan ke konsol menggunakan `cat()`.

Praktik 2: Operasi Aritmatika pada Vektor

Pada bagian ini vektor `nilai` yang berisi `c(80, 85, 90, 95, 100)` digunakan untuk operasi aritmatika. Baris `nilai_tambah <- nilai + 5` berarti setiap elemen pada vektor `nilai` akan ditambah 5 sehingga menghasilkan 85, 90, 95, 100, dan 105. Kemudian dibuat vektor `bonus <- c(1, 2, 3, 4, 5)` dan operasi `total <- nilai + bonus` menjumlahkan elemen sesuai indeksinya, misalnya $80 + 1$, $85 + 2$, dan seterusnya sehingga hasilnya vektor baru dengan nilai total. Selanjutnya digunakan operasi logis dengan `tinggi <- nilai > 85` yang menghasilkan nilai logika `TRUE` untuk elemen yang lebih besar dari 85, dan `FALSE` untuk elemen yang tidak lebih besar dari 85.

Praktik 3: Pengindeksan dan Filtering Vektor

Program berikutnya menjelaskan cara mengakses elemen vektor pada R. Misalnya dengan `nilai[2]` akan mengakses elemen kedua dari vektor `nilai`, yaitu angka 85. Dengan `nilai[1:3]` akan mengambil elemen ke-1 sampai ke-3, sehingga hasilnya 80, 85, dan 90. Pada bagian filtering, digunakan `nilai_tinggi <- nilai[nilai > 85]` yang berarti hanya nilai yang lebih besar dari 85 saja yang akan disimpan ke dalam vektor baru `nilai_tinggi`.

2. Praktik 2

```
[15] ✓ 0 d # Membuat vektor karakter
jenis_kelamin <- c("Laki-laki", "Perempuan", "Laki-laki", "Perempuan", "Laki-laki")

# Membuat faktor tidak terurut
jk_faktor <- factor(jenis_kelamin)
cat("Faktor jenis kelamin:", jk_faktor, "\n")
cat("Level faktor:", levels(jk_faktor), "\n")
cat("Jumlah level:", nlevels(jk_faktor), "\n")

# Mengubah urutan level
jk_faktor_urut <- factor(jenis_kelamin, levels = c("Perempuan", "Laki-laki"))
cat("Level dengan urutan khusus:", levels(jk_faktor_urut), "\n")

Faktor jenis kelamin: 1 2 1 2 1
Level faktor: Laki-laki Perempuan
Jumlah level: 2
Level dengan urutan khusus: Perempuan Laki-laki

[16] ✓ 0 d # Membuat vektor untuk tingkat pendidikan
pendidikan <- c("SD", "SMP", "SMA", "SMP", "Sarjana", "SMA")

# Membuat faktor terurut
pend_faktor <- ordered(pendidikan, levels = c("SD", "SMP", "SMA", "Sarjana"))
cat("Faktor pendidikan terurut:", pend_faktor, "\n")
cat("Level faktor:", levels(pend_faktor), "\n")
cat("Apakah terurut?", is.ordered(pend_faktor), "\n")

Faktor pendidikan terurut: 1 2 3 2 4 3
Level faktor: SD SMP SMA Sarjana
Apakah terurut? TRUE

[17] ✓ 0 d # Membuat vektor karakter
jenis_kelamin <- c("Laki-laki", "Perempuan", "Laki-laki", "Perempuan", "Laki-laki")

# Membuat faktor tidak terurut
jk_faktor <- factor(jenis_kelamin)

# Konversi faktor ke karakter
cat("Faktor ke karakter:", as.character(jk_faktor), "\n")

# Mengubah level referensi
jk_relevel <- relevel(jk_faktor, ref = "Perempuan")
cat("Level setelah relevel:", levels(jk_relevel), "\n")

# Menghapus level tidak terpakai
faktor_lengkap <- factor(c("A", "B", "C", "A"))
faktor_diseder <- droplevels(faktor_lengkap)
cat("Level setelah drop:", levels(faktor_diseder), "\n")

Faktor ke karakter: Laki-laki Perempuan Laki-laki Perempuan Laki-laki
Level setelah relevel: Perempuan Laki-laki
Level setelah drop: A B C
```

Pembahasan :

B. Membuat dan Memanipulasi Faktor

Praktik 1: Membuat Faktor Tidak Terurut

Program di atas menggunakan `jenis_kelamin <- c("Laki-laki", "Perempuan", "Laki-laki", "Perempuan", "Laki-laki")` untuk membuat vektor karakter. Kemudian baris `jk_faktor <- factor(jenis_kelamin)` mengubah vektor karakter tersebut menjadi faktor tidak terurut. Ketika dicetak dengan `cat("Level faktor:", levels(jk_faktor))` akan menampilkan level yang ada yaitu "Laki-laki" dan "Perempuan". Selain itu digunakan `nlevels(jk_faktor)` untuk menghitung jumlah level. Pada baris berikutnya dibuat faktor dengan urutan khusus menggunakan `factor(jenis_kelamin, levels = c("Perempuan", "Laki-laki"))` sehingga urutan level menjadi Perempuan terlebih dahulu baru Laki-laki.

Praktik 2: Membuat Faktor Terurut

Pada bagian ini dibuat vektor pendidikan `<- c("SD", "SMP", "SMA", "SMP", "Sarjana", "SMA")`. Kemudian dibuat faktor terurut dengan `ordered(pendidikan, levels = c("SD", "SMP", "SMA", "Sarjana"))`. Perbedaan dari faktor biasa yaitu faktor terurut memiliki tingkatan yang jelas sehingga bisa dilakukan perbandingan berdasarkan urutan levelnya. Fungsi `is.ordered()` digunakan untuk mengecek apakah faktor tersebut benar-benar terurut.

Praktik 3: Operasi pada Faktor

Di bagian ini pertama kali ditunjukkan cara mengonversi faktor ke karakter dengan `as.character(jk_faktor)`. Lalu ditunjukkan cara mengubah level referensi menggunakan `relevel(jk_faktor, ref = "Perempuan")` sehingga level Perempuan dijadikan referensi utama. Terakhir digunakan `droplevels()` untuk menghapus level yang tidak terpakai, contohnya ketika sebuah faktor dibuat dari data dengan level lebih banyak daripada observasi yang ada.

3. Praktik 3

```
[18]
✓ Od

# Membuat data frame
data_siswa <- data.frame(
  Nama = c("Ali", "Budi", "Cici", "Dewi"),
  Nilai = c(80, 85, 90, 75),
  Kelas = factor(c("A", "B", "A", "B")),
  Usia = c(20, 21, 19, 20)
)

cat("Data frame siswa:\n")
print(data_siswa)

# Melihat struktur data frame
str(data_siswa)
```

Data frame siswa:

	Nama	Nilai	Kelas	Usia
1	Ali	80	A	20
2	Budi	85	B	21
3	Cici	90	A	19
4	Dewi	75	B	20

'data.frame': 4 obs. of 4 variables:
\$ Nama : chr "Ali" "Budi" "Cici" "Dewi"
\$ Nilai: num 80 85 90 75
\$ Kelas: Factor w/ 2 levels "A","B": 1 2 1 2
\$ Usia : num 20 21 19 20

```
[19]
✓ Od

# Mengakses kolom
cat("Kolom Nilai:", data_siswa$Nilai, "\n")

# Mengakses baris
cat("Baris pertama:\n")
print(data_siswa[1, ])

# Mengakses elemen tertentu
cat("Nilai Budi:", data_siswa[2, "Nilai"], "\n")

# Menambahkan kolom baru
data_siswa$lulus <- data_siswa$Nilai >= 80
cat("Data frame dengan kolom lulus:\n")
print(data_siswa)

# Menggunakan subset() untuk filtering
data_lulus <- subset(data_siswa, lulus == TRUE)
cat("Siswa yang lulus:\n")
print(data_lulus)
```

Kolom Nilai: 80 85 90 75
Baris pertama:

	Nama	Nilai	Kelas	Usia
1	Ali	80	A	20

Nilai Budi: 85
Data frame dengan kolom lulus:

	Nama	Nilai	Kelas	Usia	lulus
1	Ali	80	A	20	TRUE
2	Budi	85	B	21	TRUE
3	Cici	90	A	19	TRUE
4	Dewi	75	B	20	FALSE

Siswa yang lulus:

	Nama	Nilai	Kelas	Usia	lulus
1	Ali	80	A	20	TRUE
2	Budi	85	B	21	TRUE
3	Cici	90	A	19	TRUE

```
[18] ✓ Od # Membuat data frame
data_siswa <- data.frame(
  Nama = c("Ali", "Budi", "Cici", "Dewi"),
  Nilai = c(80, 85, 90, 75),
  Kelas = factor(c("A", "B", "A", "B")),
  Usia = c(20, 21, 19, 20)
)
cat("Data frame siswa:\n")
print(data_siswa)

# Melihat struktur data frame
str(data_siswa)

Data frame siswa:
  Nama Nilai Kelas Usia
1 Ali    80     A   20
2 Budi   85     B   21
3 Cici   90     A   19
4 Dewi   75     B   20
'data.frame':   4 obs. of  4 variables:
 $ Nama : chr  "Ali" "Budi" "Cici" "Dewi"
 $ Nilai: num  80 85 90 75
 $ Kelas: Factor w/ 2 levels "A","B": 1 2 1 2
 $ Usia : num  20 21 19 20

[19] ✓ Od # Mengakses kolom
cat("Kolom Nilai:", data_siswa$Nilai, "\n")

# Mengakses baris
cat("Baris pertama:\n")
print(data_siswa[1, ])

# Mengakses elemen tertentu
cat("Nilai Budi:", data_siswa[2, "Nilai"], "\n")

# Menambahkan kolom baru
data_siswa$Lulus <- data_siswa$Nilai >= 80
cat("Data frame dengan kolom Lulus:\n")
print(data_siswa)

# Menggunakan subset() untuk filtering
data_lulus <- subset(data_siswa, Lulus == TRUE)
cat("Siswa yang lulus:\n")
print(data_lulus)

Kolom Nilai: 80 85 90 75
Baris pertama:
  Nama Nilai Kelas Usia
1 Ali    80     A   20
Nilai Budi: 85
Data frame dengan kolom Lulus:
  Nama Nilai Kelas Usia Lulus
1 Ali    80     A   20  TRUE
2 Budi   85     B   21  TRUE
3 Cici   90     A   19  TRUE
4 Dewi   75     B   20  FALSE
Siswa yang lulus:
  Nama Nilai Kelas Usia Lulus
1 Ali    80     A   20  TRUE
2 Budi   85     B   21  TRUE
3 Cici   90     A   19  TRUE
```

Pembahasan : C. Membuat dan Memanipulasi Data Frame

Praktik 1: Membuat Data Frame
Program menggunakan `data.frame()` untuk membuat tabel dengan kolom Nama, Nilai, Kelas, dan Usia. Kolom Kelas didefinisikan sebagai faktor sehingga level yang ada akan otomatis terbentuk dari data. Fungsi `str(data_siswa)` digunakan untuk melihat struktur data frame, termasuk tipe data tiap kolom.

Praktik 2: Mengakses dan Memodifikasi Data Frame
Pada bagian ini ditunjukkan cara mengakses kolom dengan `$`, misalnya `data_siswa$Nilai` akan menampilkan semua nilai dari kolom Nilai. Untuk mengakses baris digunakan notasi indeks `[1, ]` yang berarti mengambil baris pertama. Untuk mengakses elemen tertentu digunakan kombinasi indeks baris dan nama kolom misalnya `data_siswa[2, "Nilai"]`. Kemudian ditambahkan kolom baru Lulus yang berisi hasil logika `Nilai >= 80`. Dengan `subset()` dilakukan filtering siswa yang hanya lulus.

Praktik 3: Manipulasi dengan dplyr
 Bagian ini menggunakan library dplyr. Program filter(Nilai >= 85) akan menampilkan hanya baris dengan nilai di atas 85. Program group_by(Kelas) %>% summarise(Rata_rata = mean(Nilai)) akan menghitung rata-rata nilai berdasarkan tiap kelas.

Praktik 4: Menggabungkan Data Frame
 Program membuat data tambahan dengan kolom Nama dan Peminatan. Dengan fungsi merge(data_siswa, data_tambahan, by = "Nama", all.x = TRUE) kedua data frame digabungkan berdasarkan kolom Nama, sehingga informasi tambahan akan ikut ditambahkan pada siswa yang ada.

4. Praktik 4

```
[22]
✓ 0 d

# Membuat matriks 2x3
matriks <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3, byrow = FALSE)
cat("Matriks 2x3 (isi per kolom):\n")
print(matriks)

# Membuat matriks dengan pengisian per baris
matriks_byrow <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3, byrow = TRUE)
cat("Matriks 2x3 (isi per baris):\n")
print(matriks_byrow)

# Menambahkan nama baris dan kolom
rownames(matriks) <- c("Baris1", "Baris2")
colnames(matriks) <- c("Kolom1", "Kolom2", "Kolom3")
cat("Matriks dengan nama:\n")
print(matriks)
```

```
Matrics 2x3 (isi per kolom):
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
Matrics 2x3 (isi per baris):
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
Matrics dengan nama:
      Kolom1 Kolom2 Kolom3
Baris1     1     3     5
Baris2     2     4     6
```

```
[23]
✓ 0 d

# Transpos matriks
cat("Transpos matriks:\n")
print(t(matriks))

# Perkalian matriks
matriks2 <- matrix(c(1, 0, 0, 1, 0, 0), nrow = 3, ncol = 2)
hasil_kali <- matriks %*% matriks2
cat("Hasil perkalian matriks:\n")
print(hasil_kali)

# Invers matriks (khusus matriks persegi)
matriks_persegi <- matrix(c(4, 3, 3, 2), nrow = 2, ncol = 2)
invers <- solve(matriks_persegi)
cat("Invers matriks persegi:\n")
print(invers)
```

```
Transpos matriks:
      Baris1 Baris2
Kolom1     1     2
Kolom2     3     4
Kolom3     5     6
Hasil perkalian matriks:
      [,1] [,2]
Baris1     1     1
Baris2     2     2
Invers matriks persegi:
      [,1] [,2]
[1,]    -2     3
[2,]     3    -4
```

```
[24]
✓ Od

# Membuat array 3D (2x3x2)
array_data <- array(1:12, dim = c(2, 3, 2))
cat("Array 3D:\n")
print(array_data)

# Menambahkan nama dimensi
dimnames(array_data) <- list(
  Baris = c("B1", "B2"),
  Kolom = c("K1", "K2", "K3"),
  Dimensi = c("D1", "D2")
)
cat("Array dengan nama dimensi:\n")
print(array_data)
```

```
Array 3D:
, , 1
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

Array dengan nama dimensi:
, , Dimensi = D1
      Kolom
Baris K1 K2 K3
B1    1  3  5
B2    2  4  6

, , Dimensi = D2
      Kolom
Baris K1 K2 K3
B1    7  9 11
B2    8 10 12
```

```
[25]
✓ Od

# Mengakses elemen matriks
cat("Elemen [1,2] matriks:", matriks[1, 2], "\n")

# Mengakses baris/kolom
cat("Baris pertama:\n")
print(matriks[1, ])
cat("Kolom kedua:\n")
print(matriks[, 2])

# Mengakses elemen array
cat("Elemen [1,2,1] array:", array_data[1, 2, 1], "\n")

# Memodifikasi elemen
matriks[1, 1] <- 10
cat("Matriks setelah modifikasi:\n")
print(matriks)
```

```
Elemen [1,2] matriks: 3
Baris pertama:
Kolom1 Kolom2 Kolom3
1      3      5
Kolom kedua:
Baris1 Baris2
3      4
Elemen [1,2,1] array: 3
Matriks setelah modifikasi:
      Kolom1 Kolom2 Kolom3
Baris1   10     3     5
Baris2    2     4     6
```

Pembahasan : D. Membuat dan Memanipulasi Matriks dan Array

Praktik 1: Membuat Matriks

Program `matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)` menghasilkan matriks 2x3. Dengan `byrow = TRUE` isi matriks diisi per baris. Baris berikutnya menambahkan nama baris dengan `rownames()` dan nama kolom dengan `colnames()`.

Praktik 2: Operasi Matriks

Menggunakan `t(matriks)` program menampilkan transpos dari matriks. Lalu dibuat matriks `matriks2` dan dilakukan perkalian matriks dengan `%*%`. Untuk invers digunakan `solve(matriks_persegi)` yang hanya berlaku untuk matriks persegi.

Praktik 3: Membuat Array

Program `array(1:12, dim = c(2, 3, 2))` membuat array tiga dimensi berisi angka 1 sampai 12 dengan ukuran 2x3x2. Nama dimensi ditambahkan dengan `dimnames()`. Elemen array dapat diakses dengan tiga indeks [baris, kolom, dimensi].

Praktik 4: Mengakses dan Memodifikasi Matriks/Array

Program `matriks[1, 2]` mengakses elemen pada baris 1 kolom 2. Baris lengkap bisa diakses dengan `matriks[1, ]` dan kolom lengkap dengan `matriks[, 2]`. Program juga menampilkan cara memodifikasi isi matriks misalnya `matriks[1, 1] <- 10`.

5. Praktik 5

```
[26] ✓ 0 d # Membuat list dengan elemen heterogen
data_list <- list(
  Nama = c("Ali", "Budi", "Cici"),
  Nilai = c(80, 85, 90),
  Kelas = factor(c("A", "B", "A")),
  Matriks = matrix(1:4, nrow = 2, ncol = 2)
)
cat("List data:\n")
print(data_list)

# Struktur list
str(data_list)

List data:
 $Nama
 [1] "Ali" "Budi" "Cici"

 $Nilai
 [1] 80 85 90

 $Kelas
 [1] A B A
 Levels: A B

 $Matriks
      [,1] [,2]
 [1,]    1    3
 [2,]    2    4

List of 4
 $ Nama : chr [1:3] "Ali" "Budi" "Cici"
 $ Nilai : num [1:3] 80 85 90
 $ Kelas : Factor w/ 2 levels "A","B": 1 2 1
 $ Matriks: int [1:2, 1:2] 1 2 3 4

[27] ✓ 0 d # Mengakses dengan nama
cat("Nama:", data_list$Nama, "\n")

# Mengakses dengan indeks
cat("Nilai (indeks 2):", data_list[[2]], "\n")

# Mengakses sublist
sublist <- data_list[1:2]
cat("Sublist (Nama dan Nilai):\n")
print(sublist)

# Mengakses elemen dalam matriks
cat("Elemen matriks [1,2]:", data_list$Matriks[1, 2], "\n")

Nama: Ali Budi Cici
Nilai (indeks 2): 80 85 90
Sublist (Nama dan Nilai):
 $Nama
 [1] "Ali" "Budi" "Cici"

 $Nilai
 [1] 80 85 90

Elemen matriks [1,2]: 3
```

[28]
✓ 0 d

```
# Menambahkan elemen baru
data_list$Usia <- c(20, 21, 19)
cat("List setelah menambah Usia:\n")
print(data_list)

# Mengubah elemen
data_list$Nama[2] <- "Bambang"
cat("List setelah mengubah Nama[2]:\n")
print(data_list)

# Menghapus elemen
data_list$Matriks <- NULL
cat("List setelah menghapus Matriks:\n")
print(data_list)
```

```
→ List setelah menambah Usia:
$Nama
[1] "Ali" "Budi" "Cici"

$Nilai
[1] 80 85 90

$Kelas
[1] A B A
Levels: A B

$Matriks
      [,1] [,2]
[1,]    1    3
[2,]    2    4

$Usia
[1] 20 21 19

List setelah mengubah Nama[2]:
$Nama
[1] "Ali" "Bambang" "Cici"

$Nilai
[1] 80 85 90

$Kelas
[1] A B A
Levels: A B

$Matriks
      [,1] [,2]
[1,]    1    3
[2,]    2    4

$Usia
[1] 20 21 19

List setelah menghapus Matriks:
$Nama
[1] "Ali" "Bambang" "Cici"

$Nilai
[1] 80 85 90

$Kelas
[1] A B A
Levels: A B

$Usia
[1] 20 21 19
```

```
[29] ✓ 0 d
# Membuat list kedua
list_tambahan <- list(
  Jurusan = c("Matematika", "Fisika", "Kimia"),
  Status = c(TRUE, FALSE, TRUE)
)

# Menggabungkan list
list_gabung <- c(data_list, list_tambahan)
cat("List gabungan:\n")
print(list_gabung)
```

```
→ List gabungan:
$Nama
[1] "Ali"      "Bambang" "Cici"

$Nilai
[1] 88 85 90

$Kelas
[1] A B A
Levels: A B

$Usia
[1] 20 21 19

$Jurusan
[1] "Matematika" "Fisika"      "Kimia"

$Status
[1] TRUE FALSE TRUE
```

Pembahasan: E. Membuat dan Memanipulasi List

Praktik 1: Membuat List

Program `list()` digunakan untuk membuat list heterogen yang berisi vektor, faktor, dan matriks. List dapat berisi elemen dengan nama misalnya Nama, Nilai, Kelas. Fungsi `str(data_list)` akan menampilkan struktur list.

Praktik 2: Mengakses Elemen List

Program `data_list$Nama` mengakses elemen bernama Nama. Dengan `data_list[[2]]` mengakses elemen berdasarkan indeks kedua. Jika menggunakan `[1:2]` maka akan menghasilkan sublist. Elemen dalam matriks di dalam list juga bisa diakses misalnya `data_list$Matriks[1, 2]`.

Praktik 3: Memodifikasi List

Program `data_list$Usia <- c(20, 21, 19)` menambahkan elemen baru. Elemen bisa diubah misalnya `data_list$Nama[2] <- "Bambang"`. Elemen juga bisa dihapus dengan memberi nilai NULL misalnya `data_list$Matriks <- NULL`.

Praktik 4: Menggabungkan List

Program membuat list kedua bernama `list_tambahan`. Dengan `c(data_list, list_tambahan)` kedua list digabungkan sehingga menghasilkan list lebih panjang yang berisi semua elemen dari kedua list.

Latihan

1. Latihan 1

```
[31] ✓ d #Lakukan latihan berikut di Google Colab untuk memperkuat pemahaman tentang vektor:

# 1. Buat vektor numerik berisi 6 skor ujian
skor <- c(70, 75, 80, 85, 90, 95)
skor

# 2. Tambahkan 10 ke setiap elemen
skor_plus10 <- skor + 10
skor_plus10

# 3. Filter skor di atas 80
skor_diatas80 <- skor[skor > 80]
skor_diatas80

# 4. Buat vektor karakter berisi 5 nama siswa
nama <- c("Andi", "Budi", "Citra", "Dewi", "Eko")
nama
```

```
70 75 80 85 90 95
80 85 90 95 100 105
85 90 95
'Andi' 'Budi' 'Citra' 'Dewi' 'Eko'
```

Pembahasan :

Latihan Vektor

Program membuat vektor skor ujian dengan `c(70, 75, 80, 85, 90, 95)`. Kemudian operasi skor + 10 akan menambahkan 10 ke setiap elemen vektor. Filtering dilakukan dengan `skor[skor > 80]` yang menampilkan hanya skor di atas 80. Lalu dibuat vektor karakter nama lima siswa misalnya `c("Ali", "Budi", "Cici", "Dewi", "Eka")`.

2. Latihan 2

```
[ ] # 1. Vektor kota
kota <- c("Jakarta", "Bandung", "Surabaya", "Jakarta", "Medan", "Bandung")
kota

# 2. Ubah menjadi faktor tidak terurut
kota_faktor <- factor(kota)
kota_faktor
levels(kota_faktor)
nlevels(kota_faktor)

# 3. Faktor terurut untuk tingkat kepuasan
kepuasan <- factor(
  c("Sangat Tidak Puas", "Puas", "Tidak Puas", "Sangat Puas"),
  levels = c("Sangat Tidak Puas", "Tidak Puas", "Puas", "Sangat Puas"),
  ordered = TRUE
)
kepuasan

# 4. Data frame dengan Kota dan Populasi
populasi <- c(11000000, 2500000, 3000000, 11000000, 2500000, 5000000)
df_kota <- data.frame(Kota = kota_faktor, Populasi = populasi)
df_kota
```

Pembahasa : Latihan Faktor

Program membuat vektor karakter enam kota lalu diubah menjadi faktor tidak terurut dengan `factor()`. Level faktor ditampilkan dengan `levels()` dan jumlah level dengan `nlevels()`. Kemudian dibuat faktor terurut dengan

ordered() untuk tingkat kepuasan, misalnya dengan level dari *Sangat Tidak Puas* sampai *Sangat Puas*. Terakhir dibuat data frame dengan kolom Kota sebagai faktor dan Populasi sebagai angka.

3. Latihan 3

```
[ ] # 1. Data frame siswa
Nama <- c("Andi", "Budi", "Citra", "Dewi", "Eko")
Skor_Matematika <- c(85, 70, 90, 95, 75)
Skor_Fisika <- c(80, 65, 85, 90, 70)
Jurusan <- factor(c("IPA", "IPS", "IPA", "IPA", "IPS"))

df_siswa <- data.frame(Nama, Skor_Matematika, Skor_Fisika, Jurusan)
df_siswa

# 2. Tambahkan kolom Rata_rata
df_siswa$Rata_rata <- (df_siswa$Skor_Matematika + df_siswa$Skor_Fisika) / 2
df_siswa

# 3. Filter siswa dengan rata-rata > 80 (pakai dplyr)
library(dplyr)
df_siswa %>% filter(Rata_rata > 80)
```

Pembahasan : Latihan Data Frame

Program membuat data frame berisi 5 siswa dengan kolom Nama, Skor_Matematika, Skor_Fisika, dan Jurusan. Lalu dibuat kolom baru Rata_rata dengan menghitung rata-rata dari dua skor. Filtering dilakukan menggunakan filter(Rata_rata > 80) dari package dplyr.

B. PEMBAHASAN TUGAS

1. Tugas 1

```
[ ] # 1. Matriks 3x2 berisi angka 1-6 (byrow = TRUE untuk isi per baris)
m <- matrix(1:6, nrow = 3, ncol = 2, byrow = TRUE)
m

# 2. Tambahkan nama baris & kolom
rownames(m) <- c("Baris1", "Baris2", "Baris3")
colnames(m) <- c("Kolom1", "Kolom2")
m

# 3. Hitung transpose dan rata-rata per kolom
t_m <- t(m)
t_m
colMeans(m)

# 4. Array 3D 2x2x2 berisi angka 1-8
arr <- array(1:8, dim = c(2,2,2))
arr
```

Pembahasan : Tugas Matriks dan Array

Program membuat matriks 3x2 berisi angka 1 sampai 6 dengan byrow = TRUE. Nama baris ditambahkan misalnya "Baris1", "Baris2", "Baris3", dan nama kolom "Kolom1", "Kolom2". Transpos dihitung dengan t(matriks) dan rata-rata per kolom dihitung dengan colMeans(matriks). Array 3D dibuat dengan array(1:8, dim = c(2, 2, 2)).

2. Tugas 2

```
# 1. List berisi vektor skor, nama, dan kategori (faktor)
skor <- c(70, 75, 80, 85, 90)
nama <- c("Andi", "Budi", "Citra", "Dewi", "Eko")
kategori <- factor(c("A", "B", "A", "C", "B"))

my_list <- list(Skor = skor, Nama = nama, Kategori = kategori)
my_list

# 2. Tambahkan elemen matriks 2x2
mat <- matrix(1:4, nrow=2)
my_list$Matriks <- mat
my_list

# 3. Akses dan ubah elemen kedua dari vektor Nama
my_list$Nama[2]
my_list$Nama[2] <- "Bayu"
my_list$Nama
```

Pembahasan : Tugas List

Program membuat list dengan tiga elemen yaitu vektor skor, vektor nama, dan faktor kategori. Lalu ditambahkan elemen matriks 2x2 ke dalam list. Akses elemen kedua dari vektor Nama dilakukan dengan `data_list$Nama[2]`, lalu diubah nilainya dengan assignment operator `<-`.